

Concurrency and Resilience

Concurrency is about handling many things at once; resilience is about staying up when things fail. Both show up in every production service, and they are related: concurrency creates new failure modes, and resilience is what absorbs them.

1. What is concurrency?

Concurrency is a program **making progress on multiple tasks at once** by interleaving them, so several things are in flight even if they are not literally running at the same instant. It is about structure: dealing with many things at a time.

2. What is resilience?

Resilience is a system's ability to **keep working, or degrade gracefully, when things fail**: a dependency is down, the network drops, or load spikes. It is about surviving failure rather than assuming it will not happen.

3. Why are concurrency and resilience both important, and how are they related?

Concurrency gives you **throughput**, handling many requests at once; resilience keeps that throughput **trustworthy** when parts fail. They are related because concurrency creates new failure modes, races, contention, partial failures, and resilience is what absorbs them. A fast system that falls over under load is useless, and a reliable system that handles one request at a time does not scale. You need both.

4. Can a highly concurrent system still be unreliable? Why?

Yes, easily. Concurrency only means it handles load; it says nothing about what happens when a dependency times out or a node dies. Worse, high concurrency often makes failures **bigger and faster**: more in-flight requests to lose, more retries to pile on, more contention to deadlock. Without resilience, concurrency just fails harder.

5. What are the main goals of a resilient system?

Stay **available** and **correct** under failure, **contain** a failure so it does not cascade, **degrade gracefully** rather than crash, and **recover** on its own once the fault clears.

6. What causes systems to become unavailable, and what are common failure scenarios?

The usual culprits: **overload** (more traffic than capacity), a **failed or slow dependency**, a **network partition** or dropped connection, **resource exhaustion** (threads, connections, memory), a **bad deploy**, and **cascading failure** where one slow service backs up its callers. The nastiest is not a hard crash but a **slow dependency**, because it ties up resources while still looking alive.

7. What is fault tolerance?

Fault tolerance is a system continuing to work **correctly even when a component fails**, by having redundancy or a fallback so no single failure takes it down. It is tolerating a fault, not just detecting it.

8. What is graceful degradation?

Graceful degradation is **shedding non-essential functionality to keep the core working** under stress: serve a cached or partial result, hide a slow feature, return a default. The user gets a lesser experience instead of an error page.

9. Concurrency concepts and comparisons.

Comparison	The difference
Concurrency v/s Parallelism	Interleaving tasks so many are in progress, even on one core; v/s running tasks at the same instant on multiple cores
Process v/s Thread	Own isolated memory, heavier; v/s shares the process's memory, lighter
CPU-bound v/s I/O-bound	Limited by compute, so add cores and parallelism; v/s limited by waiting on IO, so use async and more concurrency
Synchronous v/s Asynchronous	The caller waits for the result; v/s the caller continues and is notified later
Blocking v/s Non-blocking	The thread parks until the operation completes; v/s it returns at once and the work finishes elsewhere
Sequential v/s Concurrent	One task fully, then the next; v/s tasks overlap in progress

10. Thread safety, shared mutable state, immutability, and race conditions.

Thread safety means code behaves correctly when several threads use it at once. The root cause of most concurrency bugs is **shared mutable state**, data that more than one thread can read and write. **Immutable objects**, which never change after construction, are inherently thread-safe and can be shared freely, which is why immutability is the easiest fix; a **mutable object** shared across threads is the thing you have to guard. A **race condition** is when the result depends on the timing of threads, the classic being a read-modify-write like `count++` where two threads both read the old value.

11. Why are timeouts important, and how do you choose the value?

A timeout **bounds how long you wait** for something that may never answer, so a slow or dead dependency does not tie up your threads and connections forever. Without one, a single hung call can exhaust a pool and take the whole service down. Choose the value from the dependency's **normal latency plus headroom** (roughly its p99 plus a margin), not a round guess.

12. What happens if timeouts are too long or too short?

Too long, and you hold threads and connections open during an outage, which is exactly when you can least afford it, so a slow dependency drags you down with it. **Too short**, and you kill healthy-but-slow requests, turn success into failure, and trigger needless retries that add load. The value has to sit above normal latency but well below the point where waiting is pointless.

13. Connection v/s read timeout, and API v/s database timeout.

Connection timeout caps how long to wait to **establish** the connection; **read (or socket) timeout** caps how long to wait for **data once connected**. They are separate because a fast connect can still hang on a slow response. For **API v/s database timeouts**, set each to that dependency's own latency profile, and make sure the **caller's timeout is longer than the callee's**, so the inner call fails first and you get its real error instead of a vague outer timeout.

14. Timeout propagation and end-to-end budgeting.

In a chain of services a **deadline should propagate**: the caller passes down "you have 200 ms left", so each hop knows the remaining budget and does not start work that cannot finish in time. **End-to-end budgeting** means the total timeout is divided across the hops, each getting a slice, rather than every hop independently using the full timeout, which would let the total blow far past the user's patience.

15. Why retries, when to retry, and when never to retry.

Retries recover from **transient** failures: a blip, a brief timeout, a momentarily overloaded node. Retry when the failure is likely temporary and the operation is **idempotent** (safe to repeat). **Never retry** a non-idempotent operation that may have already succeeded (you would duplicate it), a client error (a 4xx like a bad request, it will just fail again), or a dependency that is clearly down (you only add load).

16. Retry strategies: fixed, exponential backoff, and backoff with jitter.

- **Fixed**: wait a constant gap between tries. Simple, but many clients retry in lockstep.
- **Exponential backoff**: double the wait each time (1s, 2s, 4s), so you back off fast when the dependency is struggling.
- **Backoff with jitter**: exponential plus a random offset, and this is the one you want in production.

Jitter matters because without it every failed client retries at the same instants, creating synchronized waves that hammer a recovering service. The random offset spreads them out.

17. Controlling retries: max attempts, retry budget, storms, and amplification.

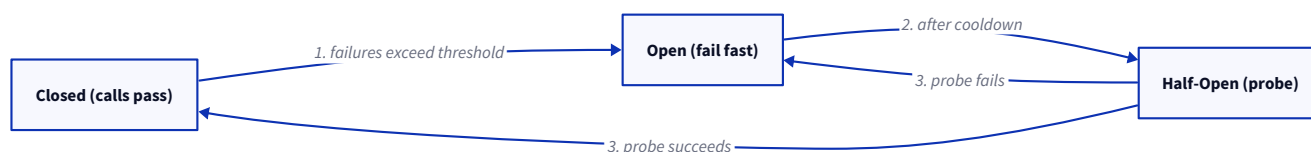
Cap the **maximum attempts** so a call cannot retry forever, and use a **retry budget** (retries allowed only up to a small fraction of total requests) so retries never dominate traffic. Without these you get a **retry storm**: a struggling service gets more load exactly when it can least take it. Worse is **retry amplification across microservices**: if each of three layers retries three times, one request becomes 27 calls. The rule is to retry at **one level, usually the edge**, not at every hop.

18. Idempotency and retries.

Idempotency means doing an operation twice has the **same effect as doing it once**, and it is what makes retries safe. If a retry might duplicate a charge or an order, you cannot retry freely. Among **HTTP methods**, GET, PUT, and DELETE are idempotent; POST is not. For a non-idempotent operation, use an **idempotency key**: the client sends a unique id, the server records it, and a repeat with the same key returns the first result instead of doing the work again. That is how you handle **duplicate requests** safely.

19. What is the circuit breaker pattern, and what problem does it solve?

A circuit breaker **stops calling a failing dependency** so you fail fast instead of piling up doomed calls. It has three states: **Closed** (calls pass), **Open** (calls fail immediately without trying), and **Half-Open** (after a cooldown, a few probe calls test whether it has recovered). It solves **cascading failure**: without it, a slow dependency ties up every caller's threads until the whole system stalls; the breaker cuts the call quickly and lets callers fall back.



20. Circuit breaker v/s retry, when together, and when not.

Retry assumes the failure is transient and tries again; a **circuit breaker** assumes the dependency is down and stops trying. They are complementary, so you **use them together**: retry the odd blip, but trip the breaker once failures are sustained, with the breaker sitting outside the retry so retries cannot become a storm. **Do not use a breaker** where each call is independent and one failure says nothing about the next, or where failing fast is worse than waiting and there is no fallback.

21. What is the bulkhead pattern, and why does it matter?

A bulkhead **isolates resources into separate pools** so a failure in one cannot sink the whole ship (the name is from a ship's watertight compartments). For example, give each downstream dependency its own thread pool or connection pool, so if one dependency hangs and exhausts its pool, the others keep serving. It **contains** resource exhaustion to a single compartment.

22. Rate limiting and load shedding.

Both protect a service from being overwhelmed. **Rate limiting** caps how many requests a client, or the system, may make in a window, so no single caller can swamp you. **Load shedding** goes further: under real overload the service **deliberately rejects some requests** (usually the lowest priority) to protect the rest, rather than slowing to a crawl and failing everything. Rate limiting is the steady-state guard; load shedding is the emergency valve. (Rate limiting has its own cheat sheet for the deeper mechanics.)

23. What is a fallback, and when should you use it?

A fallback is **what you return when the real call fails**, so the user gets something useful instead of an error. Use it whenever a degraded answer beats no answer. The common kinds:

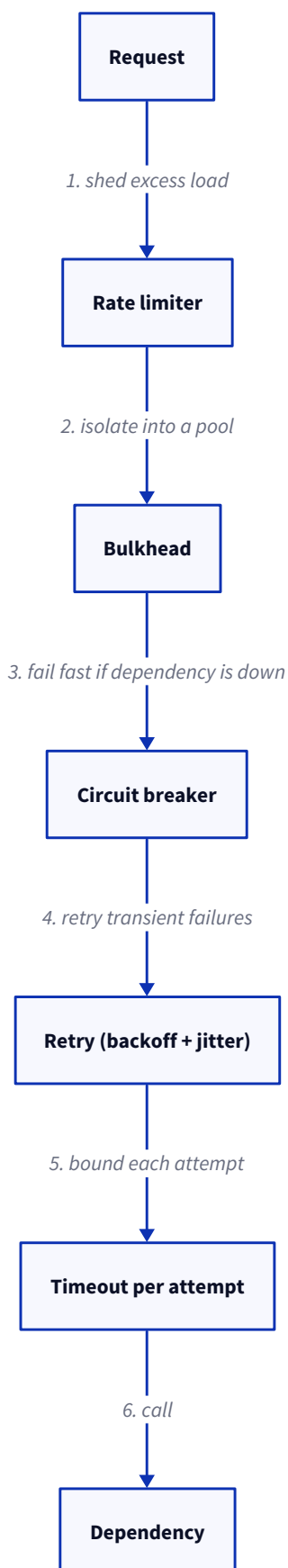
- **Static fallback**: a fixed safe value ("recommendations unavailable right now").
- **Cached fallback**: the last known good value from a cache.
- **Default response**: a sensible default, an empty list or a zero count.

A fallback pairs naturally with a circuit breaker: when the breaker is open, serve the fallback.

24. How would you combine retries, timeouts, circuit breakers, bulkheads, and rate limiting into a coherent resilience architecture?

The way I think about it is as layers wrapping the call. At the edge, **rate limiting** sheds excess load so you never take on more than you can handle. Then a **bulkhead** puts each dependency in its own pool, so one bad

dependency cannot starve the rest. Around the actual call sits a **circuit breaker** that fails fast when the dependency is clearly down, and inside that a **retry** with backoff and jitter for the transient blips, where **each attempt is bounded by a timeout** so nothing hangs. When the breaker is open or the retries are spent, a **fallback** returns a degraded but useful answer.



The order is the point: shed load first, isolate, then fail fast, then retry, and always time-bound the individual call.

25. How would you design a payment system that stays consistent despite retries, partial failures, and network partitions?

The core tension is that a payment must happen **exactly once**, but the network only gives you at-least-once at best, so I lean hard on **idempotency**. Every payment request carries an **idempotency key**; the payment service records it, so a retry with the same key returns the original result instead of charging twice. For the multi-step flow (reserve, charge, confirm) I would use a **saga with compensating actions** rather than a distributed two-phase commit, because 2PC does not survive partitions well. I would make each step idempotent and durable, drive it with a **workflow or an outbox** so a crash resumes rather than loses state, and **reconcile** against the provider afterwards as a backstop, because across a boundary I do not control, the honest target is effectively-once, not a perfect exactly-once. Under a partition I would rather **fail closed**, decline and let the user retry, than risk a double charge.